ADAPTIVE MULTI-JUNCTION TRAFFIC GRID CONTROLLER

Final Technical Report — Task 6

Professional Documentation & Delivery

RTL Design • FSM Control • Adaptive Timing • Pedestrian Arbitration • Emergency Override • Verification

Document Information	Value
Project	Adaptive Multi-Junction Traffic Grid Controller (AMTGC)
Final stage	Task 6 — Professional Documentation & Delivery
RTL language	Verilog HDL
Simulation environment	ModelSim
Clocking	Single clock domain
Reset in final RTL	Active-high asynchronous reset
Traffic-density input	3-bit per junction
Verification suite	TC01–TC15
Final verification result	15/15 scenarios executed; 0 reported safety violations
Report revision	1.0

This report is based on the supplied AMTGC report, final RTL files, adaptive timing testbench, final verification testbench, Task 5 transcript, and available ModelSim waveform evidence. The purpose is to document the implemented and verified design rather than restate the original assignment proposal.

Table of Contents

1. Project Overview

The Adaptive Multi-Junction Traffic Grid Controller (AMTGC) is an RTL-based traffic management system developed around two coordinated four-way traffic junctions. Each junction uses an FSM to control North-South and East-West traffic phases, while additional control logic handles pedestrian requests, emergency operation, and traffic-density-dependent green timing.

The project was developed incrementally. Task 1 established the architectural contract and verification scenarios; later tasks converted that architecture into synthesizable Verilog, introduced adaptive timing, and performed system-level simulation. Task 6 consolidates the final implementation, verification evidence, engineering decisions, and reproducibility information.

The final design is modular: the junction controller contains traffic-control policy and adaptive timing calculation, the generic timer performs reusable counting, and the shared pedestrian arbiter resolves pedestrian access between the two junctions.

1.1 Objectives

- Control two four-way traffic junctions using deterministic FSM sequencing.
- Provide adaptive green duration based on a 3-bit traffic-density input.
- Keep adaptive timing within configurable minimum and maximum bounds.
- Provide shared pedestrian arbitration using round-robin priority.
- Provide emergency override with safe All-Red behavior.
- Maintain a single clock domain and a consistent reset strategy.
- Verify the integrated design using directed, stress, and constrained-random scenarios.

1.2 Final Engineering Scope

The final report distinguishes between the architecture originally planned in Task 1 and the RTL that was actually supplied for Task 5 verification. Where the final RTL differs from the original plan, the implemented behavior is documented as the source of truth.

2. Final System Architecture

The final architecture is hierarchical. AMTGC_TOP instantiates two parameterized junction controllers and one shared pedestrian arbiter. Each junction controller owns its traffic FSM and adaptive timing calculation and uses a generic timer instance. The top-level module distributes the common clock, reset, traffic-density inputs, pedestrian requests, and emergency override.

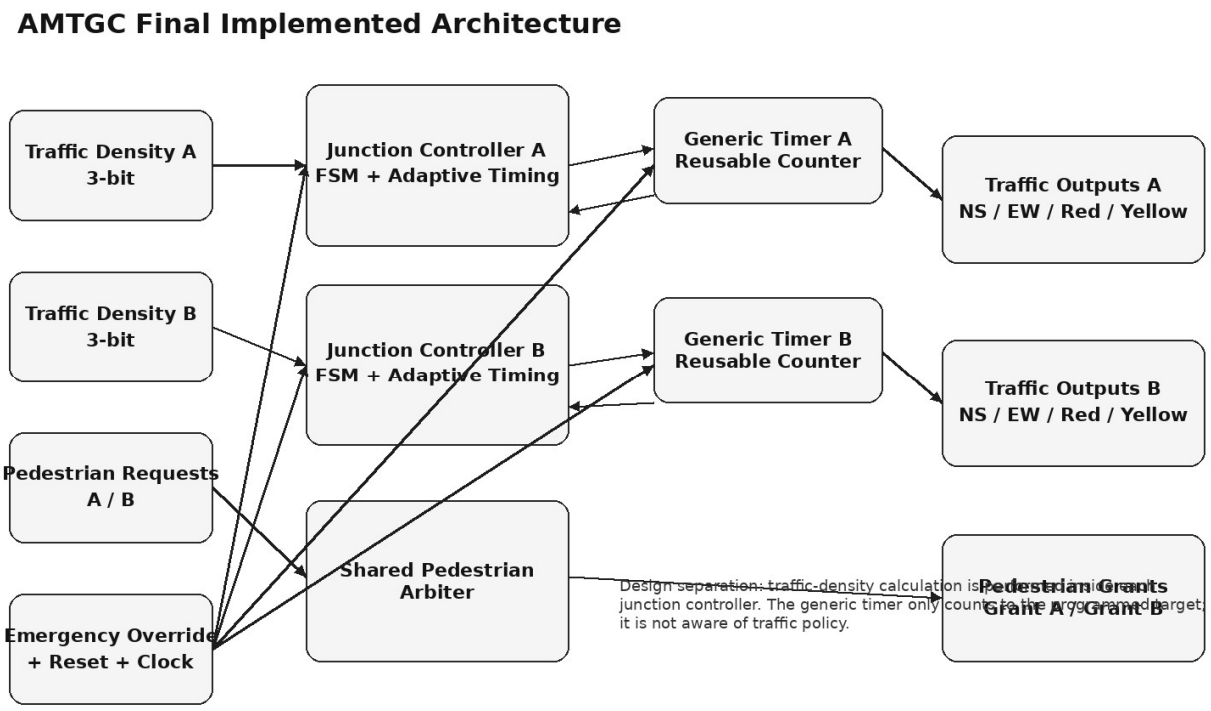

Figure 2.1 — Final implemented AMTGC architecture and module responsibilities.

2.1 Module Responsibilities

Module	Implemented responsibility
amtgc_top	Top-level integration; instantiates Junction A, Junction B, and the shared pedestrian arbiter.
junction_controller	FSM traffic sequencing, traffic-density calculation, green-time clamping, pedestrian pending/done handling, and emergency state.
generic_timer	Reusable counter that starts from a target and generates a completion pulse; no traffic policy is implemented inside the timer.
ped_arbiter	Shared round-robin arbitration between Junction A and Junction B pedestrian requests.
Testbenches	Directed adaptive timing verification and final TC01–TC15 system verification.

2.2 Architectural Separation

A central design principle is separation of policy from mechanism. The junction controller decides how long a traffic phase should last. The generic timer only counts to the target supplied by the controller. This allowed adaptive timing to be added without changing the timer's counting algorithm.

The pedestrian arbiter is shared rather than duplicated because arbitration is a system-level fairness decision. A single arbiter provides one point at which simultaneous requests are resolved and the round-robin state is maintained.

2.3 Clock and Reset

The final RTL uses one common clock signal for all modules. The clock is generated in simulation using a 10 ns period (always #5 clk = ~clk).

The final RTL uses an active-high asynchronous reset: the sequential blocks use an event control of the form posedge clk or posedge reset. This is a correction to the earlier planning document, which described a synchronous reset. The final report intentionally documents the RTL that was actually verified.

3. RTL Design

3.1 Top-Level Integration

AMTGC_TOP exposes the external system inputs: clock, reset, two 3-bit traffic-density inputs, two pedestrian requests, and the emergency override. It produces the six traffic-light outputs for each junction.

3.2 Parameterization

Junction A and Junction B use the same junction_controller RTL but are instantiated with different timing parameters. This demonstrates reuse while allowing each junction to have its own nominal green time, adaptive extension step, maximum and minimum green limits, pedestrian time, yellow time, and all-red time.

Parameter	Junction A	Junction B
GREEN_TIME	30	25
MAX_GREEN	50	45
MIN_GREEN	10	15
EXT_GREEN	5	3
PED_TIME	10	8
YELLOW_TIME	5	3
RED_TIME	2	1
COUNTER_WIDTH	8	8

3.3 Generic Timer Reuse

No traffic-density logic was added to generic_timer. Its interface remains a generic start/count_target/done interface. The controller computes the target and the timer performs the count. This preserves module reuse and limits changes to the policy layer.

The Task 5 compilation transcript confirms that amtgc_top.v, junction_controller.v, generic_timer.v, ped_arbiter.v, the adaptive testbench, and the final verification testbench all compiled successfully with zero failed compiles.

4. FSM Design

Each junction controller uses eight encoded states. Seven are operational/control states and one is the emergency state.

Encoding	State	Function
4'd0	NS_GREEN	North-South traffic green; East-West red.

4'd1	NS_YELLOW	North-South yellow; East-West red.
4'd2	ALL_RED1	Safety clearance interval before East-West green.
4'd3	EW_GREEN	East-West traffic green; North-South red.
4'd4	EW_YELLOW	East-West yellow; North-South red.
4'd5	ALL_RED2	Safety interval before returning to North-South or allowing pedestrians.
4'd6	PED_ALLOW	Pedestrian crossing with both vehicle directions red.
4'd7	EMERGENCY	Emergency state with both traffic directions forced All-Red.

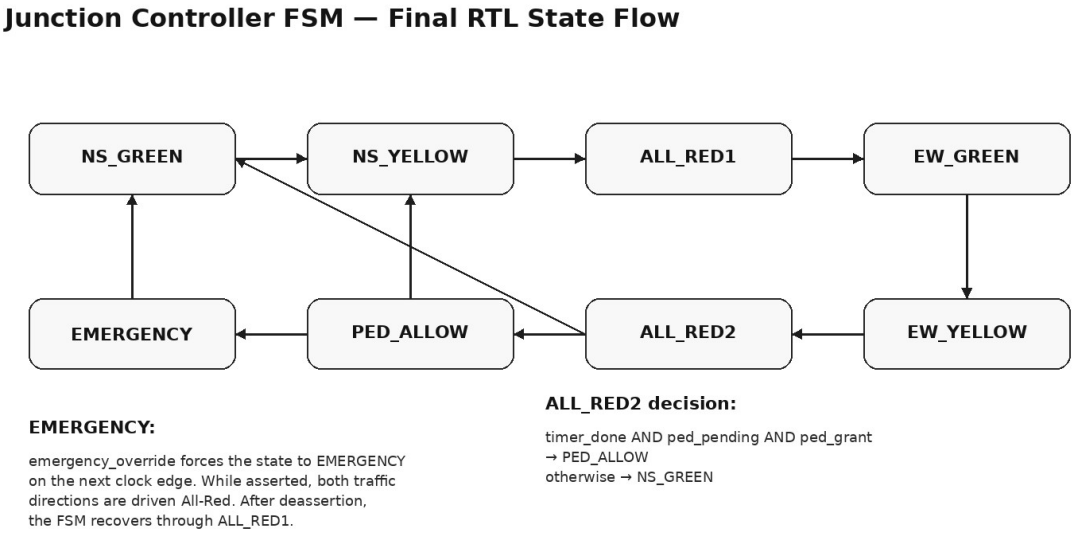

Figure 4.1 — FSM transition structure derived from the supplied final junction_controller RTL.

4.1 State Transition Logic

The normal sequence is NS_GREEN → NS_YELLOW → ALL_RED1 → EW_GREEN → EW_YELLOW → ALL_RED2. At ALL_RED2, the controller enters PED_ALLOW only when the pedestrian request is pending and the arbiter grant is active; otherwise it returns to NS_GREEN. PED_ALLOW returns to NS_GREEN after its timer completes.

When emergency_override is asserted, the state register moves to EMERGENCY on the next active clock edge. In EMERGENCY, both traffic directions are driven All-Red and the timer is not started. When the emergency input is removed, the next-state logic routes the controller through ALL_RED1 before resuming normal traffic.

5. Adaptive Traffic Timing

Adaptive timing was introduced in Task 4. The 3-bit traffic_density input ranges from 0 to 7. The junction controller calculates a raw green target using the nominal green time and an extension value:

raw_green_time = GREEN_TIME + (traffic_density × EXT_GREEN)

The result is then clamped between MIN_GREEN and MAX_GREEN. This ensures that low density cannot reduce green time below the guaranteed minimum and high density cannot extend the phase indefinitely.

5.1 Timing Mapping

Density	Junction A target	Junction B target
0	30	25
1	35	28
2	40	31
3	45	34
4	50	37
5	50 (clamped)	40
6	50 (clamped)	43
7	50 (clamped)	45 (clamped)

The mapping above follows the final RTL parameters. Junction A saturates at 50 counts at density 4 and above. Junction B saturates at 45 counts at density 7.

5.2 Why the Calculation Belongs in the Controller

Traffic density is a traffic-management policy input. The timer has no reason to know whether a target came from a fixed phase, traffic density, pedestrian timing, or another future policy. Keeping the calculation in junction_controller preserves the generic timer as reusable infrastructure.

5.3 Mid-Phase Density Change Decision

The design intentionally avoids continuously changing the active phase duration in response to a density input that changes during the phase. The controller stores the adaptive green value and updates it at the all-red boundary before the next green phase. The adaptive testbench explicitly changes Junction A density from 1 to 7 while NS_GREEN_A is active and then waits for the green phase to end, documenting that the phase should finish using the duration selected for that phase.

5.4 Maximum-Density Corner Case

The adaptive testbench also drives both junctions to 3'b111. This verifies that maximum traffic demand does not create unbounded green time and that the configured maximum remains the safety ceiling.

6. Pedestrian Arbitration

Pedestrian requests are stored in the junction controller through ped_pending. A request sets the pending flag, and the pending flag is cleared after the pedestrian phase completes. The shared ped_arbiter receives ped_pending_A and ped_pending_B and produces ped_grant_A and ped_grant_B.

6.1 Round-Robin Policy

The arbiter stores last_grant. If both requests are pending, it grants the junction opposite to the one served last. A value of 0 represents A as the last served junction, so B receives priority on the next simultaneous request; a value of 1 represents B as the last served junction, so A receives priority.

6.2 Simultaneous Requests

When both requests are pending, exactly one grant is selected. The current grant is held until the corresponding junction produces ped_done. This prevents the arbiter from changing the selected grant while a pedestrian crossing is active.

6.3 Verification Evidence

TC08 verified that simultaneous pedestrian requests were arbitrated sequentially. TC09 exercised 105 request transactions and reported successful round-robin fairness. The final transcript also reports 9 pedestrian grants served by Junction A and 3 by Junction B during the complete test suite; the difference is a result of the request pattern rather than a claim of equal counts under arbitrary workloads.

7. Emergency Override

Emergency override is treated as a high-priority safety condition. In the final junction controller, emergency_override causes the state machine to enter EMERGENCY on the next clock edge. In that state NS_red and EW_red are asserted and the timer is disabled. The timer itself is also reset while emergency_override is active.

7.1 Emergency Recovery

When emergency_override is deasserted, the EMERGENCY state transitions to ALL_RED1. This provides an explicit all-red recovery interval before normal traffic sequencing resumes.

7.2 Verification

TC11 tested emergency override while Junction A was in each of the seven non-emergency operational states: NS_GREEN, NS_YELLOW, ALL_RED1, EW_GREEN, EW_YELLOW, ALL_RED2, and PED_ALLOW. The transcript reports that every state was overridden to All-Red within one cycle.

TC12 verified recovery through an intermediate All-Red state. TC13 asserted reset during active operation, and TC14 applied repeated emergency toggles. TC15 then exercised the design under constrained-random traffic and pedestrian inputs.

8. Green-Wave Coordination

Green-wave coordination was part of the original architecture and an earlier top-level RTL variant used a GREEN_WAVE_OFFSET mechanism to delay the release of Junction B. The available Task 3 waveform evidence shows a reset_B signal and a visible offset between the junction control activity.

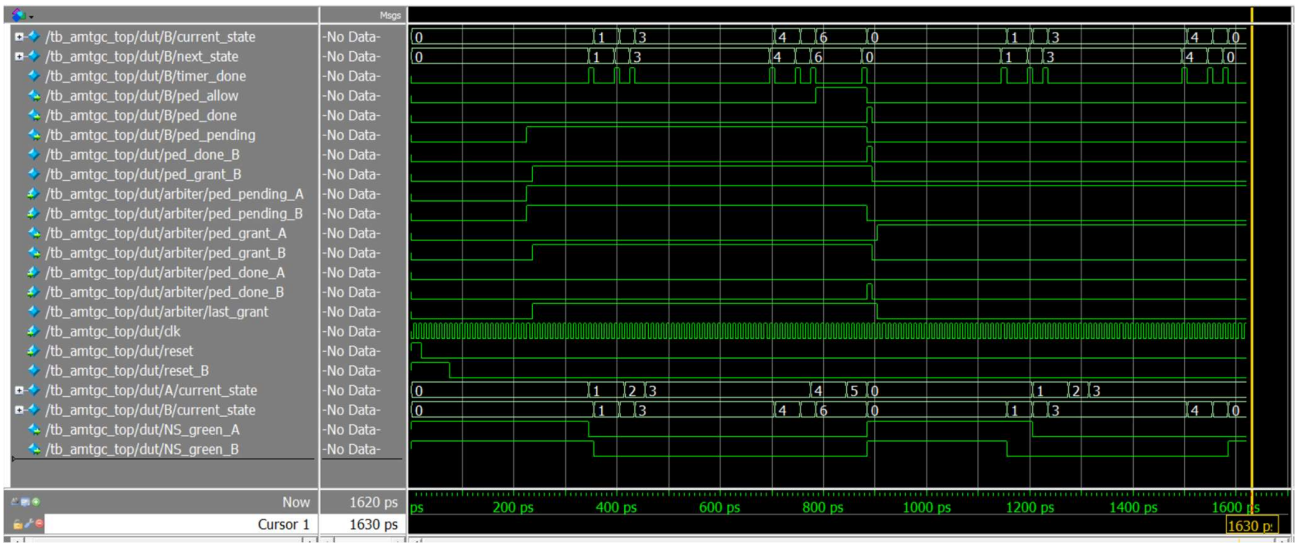

Figure 8.1 — Available Task 3 ModelSim evidence showing Junction A/B state and the B reset-offset mechanism.

However, the final Task 5 top-level RTL supplied for this report retains GREEN_WAVE_OFFSET as a parameter but does not use it in the active logic. The final top-level source directly instantiates both junction controllers with the same global reset. Therefore, this report does not claim that the final Task 5 source independently enforces a five-cycle green-wave offset.

The Task 5 TC10 test waits for Junction A's North-South green event and then Junction B's event across five density pairs: (0,1), (1,2), (2,3), (3,4), and (4,5), and reports the synchronization check as passed. Because the test does not calculate and assert the actual time difference between the two green edges, TC10 should be interpreted as event-order evidence rather than a quantitative proof of a fixed offset.

This distinction is important for professional delivery. If the assignment evaluator requires the green-wave offset to remain enforced in the final RTL, the verified offset logic should be restored and the complete Task 5 suite rerun before submission. The final RTL should not be modified without re-verification.

9. Verification Methodology

Verification was performed using ModelSim with directed tests, waveform inspection, continuous safety assertions, stress testing, and constrained-random input generation. The final transcript shows nine design/testbench compilations with zero failures before simulation.

9.1 Verification Strategy

- Module-level compilation checks for the timer, junction controller, arbiter, and top-level integration.
- Directed functional tests for reset and normal traffic sequencing.
- Adaptive timing tests including maximum-clamp behavior.
- Pedestrian tests for single, simultaneous, and repeated requests.
- Green-wave coordination checks under multiple density combinations.
- Emergency override testing across all FSM operational states.
- Mid-simulation reset and rapid emergency toggle stress.
- Constrained-random traffic density and pedestrian request activity over 120 clock cycles.
- Continuous assertion-based checks for unsafe signal combinations and system errors.

9.2 Automated Scoreboard

Metric	Reported result
Design/testbench compiles	9 successful, 0 failed
Continuous assertions checked	7,615
Safety violations/errors	0
Planned scenarios executed	15 / 15
Junction A pedestrian grants	9
Junction B pedestrian grants	3
Constrained-random cycles	120
Final verification status	PASSED

10. Simulation Results

10.1 Adaptive Timing Sweep

The adaptive timing waveform sweeps traffic density from 0 through 7 for both junctions. The dynamic green-time value increases with density and then saturates at the configured maximum.

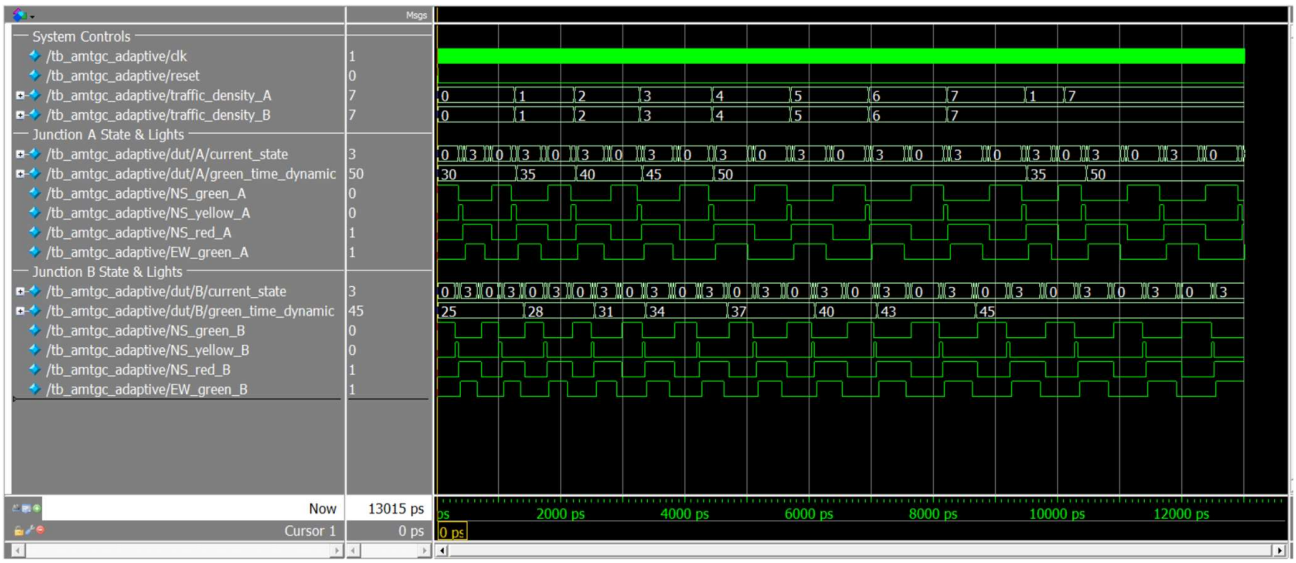

Figure 10.1 — Task 4 adaptive timing sweep. Density increases from 0 to 7 while the dynamic green targets increase and clamp at their configured limits.

10.2 Final Verification Results

ID	Scenario	Result
TC01	Power-On Reset Initialization	PASS
TC02	Normal Traffic Light Sequencing	PASS
TC03	Continuous Cyclic Operation	PASS
TC04	High Traffic Density at Junction A	PASS
TC05	High Traffic Density at Junction B / Max	PASS

	Clamp	
TC06	Single Pedestrian Request at Junction A	PASS
TC07	Single Pedestrian Request at Junction B	PASS
TC08	Simultaneous Pedestrian Arbitration	PASS
TC09	Starvation Prevention — 105 Transactions	PASS
TC10	Green-Wave Timing Across 5 Density Combinations	PASS*
TC11	Emergency Override in All 7 FSM States	PASS
TC12	Emergency Deassertion and All-Red Recovery	PASS
TC13	Active Mid-Simulation Reset	PASS
TC14	Rapid Emergency Override Glitch Stress	PASS
TC15	Constrained-Random Stability — 120 Cycles	PASS

*TC10 is reported as PASS by the testbench, but the final supplied top-level source does not actively use GREEN_WAVE_OFFSET and the test does not measure the numerical edge-to-edge offset. See Section 8 for the engineering qualification.

10.3 Key Verification Observations

- Normal sequencing progressed through NS_GREEN, NS_YELLOW, EW_GREEN, and EW_YELLOW without deadlock.
- High traffic density increased green duration and the maximum clamp was enforced.
- Single pedestrian requests were serviced at both junctions.
- Simultaneous requests were arbitrated sequentially.
- 105 repeated pedestrian transactions completed under the starvation-prevention test.
- Emergency override was exercised from all seven operational FSM states.
- Mid-simulation reset returned the system to its initialized state.
- Constrained-random testing completed 120 cycles with zero assertion failures.

11. Challenges, Bugs, and Resolutions

11.1 Pedestrian Arbitration and Persistent Pending Requests

A major debugging area was simultaneous pedestrian handling. A pending request is not necessarily a newly arriving request; it can remain asserted until the junction is actually serviced. Treating a persistent request as a new arbitration event can cause repeated or unstable arbitration behavior.

The final design therefore keeps request state inside the junction controller and uses the arbiter's last_grant state to maintain fair priority. The grant is held until ped_done is observed, after which both grants are cleared and a new arbitration decision can be made.

11.2 Adaptive Timing Stability

Continuously sampling traffic density during an active green phase could make the phase length change while the timer is already counting. The final design deliberately updates the stored dynamic green value at an all-red boundary. This gives each active green phase a stable target.

11.3 Maximum Green-Time Safety

The raw adaptive target is explicitly clamped. This prevents high traffic density from creating an indefinitely long green phase and provides a deterministic upper bound.

11.4 Documentation-to-RTL Mismatch

The original planning report described a synchronous reset, while the final RTL uses an active-high asynchronous reset. The final report corrects this discrepancy rather than carrying the planning statement forward.

11.5 Green-Wave Source Consistency

An earlier top-level variant contains the GREEN_WAVE_OFFSET counter and delayed Junction B reset. The final Task 5 top-level source supplied for verification retains the parameter but not the active offset logic. This is documented as a reproducibility caveat because Task 6 explicitly requires the repository RTL to match the verified source.

12. Design Decisions and Trade-offs

Decision	Reason	Trade-off
Shared pedestrian arbiter	Centralizes fairness and avoids duplicated arbitration logic.	Creates a shared control dependency.
Adaptive calculation in junction controller	Separates policy from generic timing mechanism.	Controller contains more timing-policy logic.
Stable green target during active phase	Prevents timing glitches from mid-phase density changes.	A sudden traffic increase waits until the next phase boundary.
Minimum/maximum green limits	Guarantees bounded operation and prevents starvation by excessive green time.	Extremely high demand cannot extend green beyond the configured cap.
Parameterized junction instances	Allows A and B to use different timing characteristics while reusing RTL.	Requires careful parameter verification.
Asynchronous active-high reset in final RTL	Provides immediate initialization response in the sequential blocks.	Reset release must still be managed carefully in a physical implementation.

13. Reproducibility and Repository Delivery

The final repository should make it possible for another engineer to clone the project, compile the RTL, run the verification suite, and inspect the evidence without verbal assistance.

13.1 Recommended Repository Structure

```
AMTGC/
│
├── rtl/
│   ├── amtgc_top.v
│   ├── junction_controller.v
│   ├── generic_timer.v
│   └── ped_arbiter.v
│
├── tb/
│   ├── tb_amtgc_top.v
│   ├── tb_amtgc_adaptive.v
│   ├──tb_amtgc_final_verification.v
│   ├──tb_generic_timer.v
│   └──tb_junction_controller.v
├── docs/
│   ├── AMTGC_Technical_Report.docx
│   ├── architecture_diagram.png
│   ├── fsm_diagram.png
│   ├── bug_log.md
│   ├── assumptions.md
│   └── self_attestation_checklist.md
│
├── sim/
│   ├── waveforms/
│   ├── screenshots/
│   └── logs/
└── README.md
```

13.2 ModelSim Reproduction

The supplied transcript shows compilation of the design and testbenches followed by simulation of tb_amtgc_final_verification. A repository README should provide the exact commands for the final filenames. A typical ModelSim flow is:

```
vlog rtl/*.v
vlog tb/tb_amtgc_final_verification.v
vsim work.tb_amtgc_final_verification
run -all
```

The exact command should be corrected to match the final repository folder structure before submission; the README should not contain commands that depend on a local Windows path.

13.3 Version Consistency Rule

The RTL committed to GitHub must be identical to the source used for the final Task 5 verification. Any change to RTL after verification must trigger recompilation and rerunning of the affected verification suite. This is particularly important for the green-wave logic discrepancy documented in Section 8.

14. Limitations and Future Improvements

- Traffic density is represented by a 3-bit abstract input rather than a physical sensor interface.
- The density-to-green-time mapping is linear and parameter-based; a real deployment could use calibrated or nonlinear traffic models.
- Verification is simulation-based. Formal verification, functional coverage metrics, and hardware-in-the-loop testing could provide additional confidence.
- Green-wave coordination should be quantitatively verified using measured edge-to-edge delay rather than only event ordering.
- The emergency input is a single global control signal. A production system could support prioritized emergency routes and multiple emergency sources.
- A real deployment would require fail-safe hardware design, signal-driver supervision, sensor validation, and physical timing constraints beyond RTL simulation.

15. Conclusion

The AMTGC project progressed from an architectural specification to a modular Verilog implementation and a system-level verification environment. The final design contains two reusable junction controllers, dedicated generic timers, adaptive green timing, a shared pedestrian arbiter, and emergency handling.

The verification environment exercised 15 planned scenarios. The supplied final transcript reports 7,615 continuous assertion checks, zero safety violations/errors, 15/15 scenarios executed, and a final PASS status. The tests cover normal traffic sequencing, adaptive timing, pedestrian service and fairness, emergency override, reset robustness, stress behavior, and constrained-random operation.

The most important engineering decisions were to keep the generic timer reusable, calculate adaptive timing inside the junction controller, bound green time with explicit minimum and maximum parameters, and use round-robin arbitration for shared pedestrian requests.

For professional delivery, the repository must preserve the exact verified RTL and make the simulation evidence reproducible. The green-wave implementation should receive particular attention before final submission because the available Task 3 waveform demonstrates an earlier offset mechanism while the

supplied final Task 5 top-level source does not actively consume GREEN_WAVE_OFFSET. Documenting this discrepancy is preferable to claiming behavior that is not present in the delivered RTL.

Appendix A — Verification Matrix

ID	Scenario	Primary evidence / check	Result
TC01	Power-On Reset	FSMs initialize to state 0	PASS
TC02	Normal sequence	NS green → yellow → EW green → EW yellow	PASS
TC03	Continuous operation	Repeated cycles without deadlock	PASS
TC04	High density A	Adaptive green extension	PASS
TC05	High density B	MAX_GREEN clamp	PASS
TC06	Pedestrian A	PED_ALLOW reached	PASS
TC07	Pedestrian B	PED_ALLOW reached	PASS
TC08	Simultaneous requests	Sequential arbitration	PASS
TC09	105 repeated transactions	Round-robin fairness	PASS
TC10	5 density pairs	Green-wave event ordering check	PASS*
TC11	Emergency in all states	All-Red within one cycle	PASS
TC12	Emergency recovery	Intermediate All-Red	PASS
TC13	Active reset	Return to initial state	PASS
TC14	Emergency glitch stress	No lockup	PASS
TC15	120 randomized cycles	Zero assertion failures	PASS

*See Section 8 for the qualification of TC10 against the supplied final top-level RTL.

Appendix B — Final RTL Configuration

Source files reviewed for this report:

- amtgc_top(4).v — final top-level supplied for Task 5 verification.
- junction_controller(3).v — final junction FSM and adaptive timing implementation.
- generic_timer(3).v — reusable timer implementation.
- ped_arbiter(3).v — final shared round-robin pedestrian arbiter.
- tb_amtgc_adaptive(2).v — Task 4 adaptive timing sweep and corner-case verification.
- tb_amtgc_final_verification(2).v — Task 5 TC01–TC15 verification suite.
- tb_amtgc_task5_transcript.log — ModelSim compile and final verification transcript.

Appendix B.1 Adaptive Timing Formula

For each junction: raw_green_time = GREEN_TIME + traffic_density × EXT_GREEN. The target is then clamped to MIN_GREEN ≤ target ≤ MAX_GREEN.

Appendix B.2 Reset Behavior

Final RTL sequential blocks use active-high asynchronous reset via posedge reset. Emergency override is handled synchronously by the junction state register but also resets the generic timer through reset || emergency_override.

Appendix C — Evidence and Submission Checklist

Deliverable	Status	Action
Final RTL source files	Required	Place exact Task 5 verified versions in /rtl
Final testbenches	Required	Place adaptive and final verification TBs in /tb
Technical report	Prepared	This document
Architecture diagram	Prepared	Included in this report; also copy PNG to /docs
FSM diagram	Prepared	Included in this report; also copy PNG to /docs
Adaptive waveform	Available	sim_tc12_adaptive_timing_sweep.png
Task 3 waveform	Available	Task_3_Test_2.png
Task 5 transcript	Available	tb_amtgc_task5_transcript.log
Bug log	Required	Create /docs/BUG_LOG.md
Assumptions document	Required	Create /docs/ASSUMPTIONS.md
Self-attestation checklist	Required	Create /docs/SELF_ATTESTATION.md
README	Required	Include exact compile/simulation commands
Demonstration video	Required	Place final MP4 in prescribed submission location
GitHub repository	Required	Verify public accessibility and final folder structure

End of report.